

20 JavaScript Interview Questions, Free Sample

JavaScript

Every question was generated from real code, run for real, and checked against the actual output before being kept. Every wrong answer is explained too, not just the right one.

How this book was built

Most interview question banks are written by hand and quietly carry the mistakes of whoever wrote them. This one was built the other way around. Every code snippet in this book was run for real, and the printed answer key is the printed output, not someone's memory of what it should be. Where two independent ways of running the same snippet disagreed, the question was dropped rather than guessed at.

Each question also explains why the three wrong answers are wrong, not only why the right one is right. That is the part most practice tests skip, and it is the part that actually teaches you something the second time you get a question wrong.

JavaScript Questions

Block 1, Types, Coercion and Equality

1. What does this print?

```
console.log('5' - 2);
```

- A. 7
- B. 3
- C. 52
- D. NaN

2. What does this print?

```
console.log(1 + 2 + 'a');
```

- A. NaNa
- B. 12a
- C. a3
- D. 3a

Block 2, Scope, Hoisting and Closures

3. What does this print?

```
let x = 'outer';  
{  
  let x = 'inner';  
  console.log(x);  
}
```

- A. undefined
- B. outer
- C. inner
- D. ReferenceError

4. What does this print?

```
let x = 'outer';  
function show(x) {  
  console.log(x);  
}  
show('inner');
```

- A. undefined
- B. inner
- C. ReferenceError
- D. outer

Block 3, this, Prototypes and Classes

5. What does this print?

```
const obj = { a: 1 };  
console.log(obj.hasOwnProperty('a'), obj.hasOwnProperty('toString'));
```

- A. true false
- B. true true
- C. false false
- D. false true

6. What does this print?

```
class Shape {  
  static count = 0;  
  constructor() {  
    Shape.count++;  
  }  
}  
class Circle extends Shape {}  
new Circle();  
new Circle();  
new Shape();  
console.log(Shape.count);
```

- A. 2
- B. 3
- C. 1
- D. 0

7. What does this print?

```
const Named = class MyClass {  
  static label() {  
    return MyClass.name;  
  }  
};  
console.log(Named.label());
```

- A. Named
- B. ReferenceError
- C. MyClass
- D. undefined

8. What does this print?

```
function Shape() {}
Shape.prototype.area = function() { return 0; };
function Circle(radius) {
  this.radius = radius;
}
Object.assign(Circle.prototype, Shape.prototype, {
  area() { return this.radius * this.radius * 3; }
});
const c = new Circle(2);
console.log(c.area());
```

- A. 12
- B. undefined
- C. 0
- D. TypeError

Block 4, Arrays and Iteration

9. What does this print?

```
console.log(['a', 'b', 'c'].reduceRight((acc, x) => acc + x));
```

- A. abc
- B. cab
- C. cba
- D. undefined

10. What does this print?

```
const [first, ...rest] = [1, 2, 3, 4];
console.log(JSON.stringify(rest));
```

- A. [2,3,4]
- B. [1,2,3,4]
- C. [3,4]
- D. [1]

Block 5, Objects, Destructuring and Immutability

11. What does this print?

```
const response = { data: { user: { id: 7 } } };
const { data: { user: { id } } } = response;
console.log(id);
```

- A. ReferenceError
- B. {"id":7}
- C. 7
- D. undefined

12. What does this print?

```
const { list: [firstItem] } = { list: ['a', 'b'] };  
console.log(firstItem);
```

- A. a
- B. ["a","b"]
- C. undefined
- D. b

13. What does this print?

```
console.log({ a: 1 } === { a: 1 });
```

- A. undefined
- B. TypeError
- C. false
- D. true

Block 6, Promises, async await and the Event Loop

14. What is the exact order of the output?

```
const trace = [];  
trace.push('start');  
setTimeout(() => trace.push('timeout'), 0);  
Promise.resolve().then(() => trace.push('microtask'));  
trace.push('sync-end');  
setTimeout(() => console.log(trace.join(' ')), 0);
```

- A. start sync-end timeout microtask
- B. start timeout microtask sync-end
- C. start sync-end microtask timeout
- D. start microtask sync-end timeout

15. What does this print?

```
Promise.resolve(1)  
  .then(async v => {  
    await null;  
    return v + 1;  
  })  
  .then(v => console.log(v));
```

- A. 2
- B. 1
- C. [object Promise]
- D. undefined

16. What does this print?

```
const trace = [];  
async function main() {  
  try {  
    await Promise.all([Promise.resolve(1), Promise.reject(new Error('mid-fail')), Promise.resolve(3)]);  
    trace.push('never');  
  } catch (e) {  
    trace.push('caught:' + e.message);  
  }  
  console.log(trace.join(' '));  
}  
main();
```

- A. caught:undefined
- B. caught:mid-fail
- C. never
- D. caught:1,mid-fail,3

Block 7, Numbers, Strings and Regular Expressions

17. What does this print?

```
console.log(5 << 2)
```

- A. 7
- B. 10
- C. 20
- D. 25

18. What does this print?

```
console.log('Hello'.at(-1))
```

- A. o
- B. H
- C. undefined
- D. olleH

Block 8, Errors, Control Flow and Sharp Edges

19. What does this print?

```
class NotFoundError extends Error {}  
class TimeoutError extends Error {}  
function handle(err) {  
  if (err instanceof NotFoundError) return 'missing';  
  if (err instanceof TimeoutError) return 'slow';  
  return 'unknown';  
}  
console.log(handle(new TimeoutError('late')));
```

- A. TimeoutError
- B. slow
- C. unknown

D. missing

20. What does this print?

```
try {  
  throw 42;  
} catch (e) {  
  console.log(e + 8);  
}
```

A. 50

B. 42

C. NaN

D. undefined8

Answers and Explanations

Organized in the same order as the questions. Each option is explained, the correct one and the three distractors.

JavaScript Answers

1. Correct answer, **B**

A. 7

Adds the two values together instead of subtracting them.

B. (correct) 3

Correct. Minus has no concatenation meaning, so it always converts both operands to numbers before subtracting.

C. 52

Assumes the minus operator concatenates strings the same way the plus operator can.

D. NaN

Assumes subtraction cannot coerce a numeric string into a number the way addition would.

Unlike plus, minus never means concatenation. It coerces both sides to numbers unconditionally, so a numeric string behaves exactly like the number it represents.

2. Correct answer, **D**

A. NaNa

Assumes the string appearing later in the expression corrupts the earlier numeric addition into an invalid number.

B. 12a

Assumes every operand becomes a string from the start because a string appears somewhere in the expression, instead of evaluating left to right until a string is introduced.

C. a3

Assumes the string operand is joined first and the numbers are appended after it.

D. (correct) 3a

Correct. The two numbers add normally first since no string has appeared yet, and the string only joins in at the last step.

Left to right evaluation means 1 + 2 completes as ordinary numeric addition before the string ever enters the picture. Only the final plus, the one touching the string, switches to concatenation.

3. Correct answer, **C**

A. undefined

Assumes the inner declaration is hoisted without its assignment, similar to a var mistake, and read before being set. The assignment happens on the same line as the declaration, so the log already sees the initialized value.

B. outer

Assumes the inner block reuses the outer binding instead of creating its own. A block introduces a new scope, so the inner let x is a completely separate variable that shadows the outer one inside the block.

C. (correct) inner

Correct. The inner block creates its own scope with a new x that shadows the outer one, so the console.log inside the block prints the inner value.

D. ReferenceError

Assumes redeclaring x inside a nested block conflicts with the outer declaration. Shadowing in a separate block is legal, only redeclaring in the exact same block scope would throw.

Curly braces alone create a new block scope for let and const declarations. A let declared inside that block shadows any variable of the same name from an outer scope for the rest of the block, without affecting the outer binding at all.

4. Correct answer, B

A. undefined

Assumes the parameter is not yet assigned when the function body runs. Parameters are initialized with the arguments passed at call time before the body executes.

B. (correct) inner

Correct. The parameter x shadows the outer let x for the entire function body, so console.log prints the argument that was passed in, inner.

C. ReferenceError

Assumes naming a parameter the same as an outer variable is a conflicting redeclaration. Parameters simply shadow outer bindings, they never clash with them.

D. outer

Assumes the function reads the outer let x instead of its own parameter. A parameter named x always creates a local binding that shadows any outer variable of the same name for the whole function body.

A function parameter creates its own binding in the function scope, independent of any variable with the same name in an enclosing scope. Inside the function, every reference to that name resolves to the parameter, never to the outer variable.

5. Correct answer, A

A. (correct) true false

Correct. a is an own property of obj, while toString is only reachable through the inherited Object.prototype, so hasOwnProperty reports it as false.

B. true true

Assumes hasOwnProperty reports true for anything reachable on the object, whether it lives directly on the object or somewhere up its prototype chain. hasOwnProperty deliberately ignores inherited properties, it only checks the object itself.

C. false false

Assumes toString somehow fails to exist on obj at all, when it is reachable through Object.prototype and calling obj.toString() would work fine. hasOwnProperty is about where a property lives, not about whether it can be accessed.

D. false true

Assumes the order is reversed, that the built in toString counts as owned while the literal property a does not. a was written directly into the object literal, making it the clearly own property, while toString only comes from the shared prototype.

hasOwnProperty checks only the properties that live directly on the object it is called on, completely ignoring the prototype chain. a was set in the object literal, so it counts as own. toString is never defined on obj itself, it is inherited from Object.prototype, so hasOwnProperty correctly reports false even though obj.toString() would still work through inheritance.

6. Correct answer, B

A. 2

Assumes creating a Circle instance does not touch the static counter defined on Shape, only direct Shape instances would. Circle has no constructor of its own, so it implicitly calls the inherited Shape constructor,

which still runs the increment.

B. (correct) 3

Correct. Every instance, whether created directly from Shape or from the subclass Circle, runs the Shape constructor body, which increments the shared static counter each time.

C. 1

Assumes only the last instantiation, the direct new Shape(), is counted. The counter accumulates across every constructor call in order, not just the final one, so all three calls add to the same running total.

D. 0

Assumes a static field declared with an initializer never actually changes after the class is defined. The initializer only sets the starting value, the constructor body runs on every instantiation and updates it from there.

A static field belongs to the class itself, and Shape.count is initialized once to 0 when the class is defined. Every time the constructor body runs, whether through new Shape() directly or through new Circle() which inherits the constructor, Shape.count++ executes and updates the one shared value. Three constructions in total bring the counter to 3.

7. Correct answer, C

A. Named

Assumes the inner name is just an alias for whatever variable the class expression was assigned to on the outside, so it would reflect the outer binding name instead. The name given in a class expression is a separate binding visible only inside the class body, it always refers to the class itself, not to the outer variable it happens to be stored in.

B. ReferenceError

Assumes referencing the internal name from a static method fails because that name is only meant for recursive references and not for general use. The internal binding created by a named class expression works like any normal reference to the class, it can be used for recursion or for anything else, including simply reading its name property.

C. (correct) MyClass

Correct. A named class expression makes its own name available inside its body only, so the static method reads MyClass, which is the class own name regardless of the outer variable it was assigned to.

D. undefined

Assumes the internal class name is not actually usable from inside the class own static methods, resulting in a missing value. A named class expression name is fully valid and readable from any method defined in that class body, static or not, exactly like an ordinary function reference.

Writing class MyClass { ... } as an expression assigned to Named creates an internal binding named MyClass that is visible only from within the class body itself, separate from the outer variable Named. Inside label, MyClass refers to the class itself, and its name property reflects that internal name, not the variable it was ultimately stored in. So Named.label() reads MyClass.name, giving the string MyClass.

8. Correct answer, A

A. (correct) 12

Correct. Object.assign copies properties left to right, so the area method from the plain object argument overwrites the one copied earlier from Shape.prototype, and that final version reads radius.

B. undefined

Assumes none of the sources properties actually get copied onto Circle.prototype, leaving area unset. Object.assign genuinely copies own enumerable properties from each source object onto the target, area does end up defined on Circle.prototype by the end of the call.

C. 0

Assumes the first area definition encountered among the sources, the one from Shape.prototype, is the one that survives on Circle.prototype. Object.assign copies each source object properties in order, later sources

overwrite properties of the same name from earlier sources, so the last area definition wins.

D. TypeError

Assumes calling the copied method later on a Circle instance is disallowed because it was attached through Object.assign rather than directly in a class or prototype assignment. A method copied onto a prototype through Object.assign behaves exactly like any other prototype method once it is there, it is called normally through instances.

Object.assign copies properties from its source arguments onto the target object in order, and when multiple sources define the same property name, the one from the later source argument wins. Here Shape.prototype.area is copied first, then overwritten by the area defined in the final plain object argument, since it comes after Shape.prototype in the argument list. Calling c.area() therefore runs that final version, computing 2 times 2 times 3.

9. Correct answer, C

A. abc

Assumes reduceRight processes the array in the same left to right order as reduce. reduceRight always folds the array starting from the last element and moving toward the first.

B. cab

Assumes the last element ends up somewhere in the middle of the result rather than at the very front. reduceRight starts its very first step with the last two elements, so the last element always leads the accumulation.

C. (correct) cba

Correct. reduceRight folds the array from right to left, so starting from "c" and appending "b" then "a" produces the string "cba".

D. undefined

Assumes reduceRight without an explicit initial value fails the same way reduce does on an empty array. The array here is not empty, so the last element simply becomes the seed and the reduction proceeds normally. *reduceRight behaves exactly like reduce except that it processes the array from the last element toward the first. Without an initial value it seeds the accumulator with the last element rather than the first, then works backward through the rest of the array.*

10. Correct answer, A

A. (correct) [2,3,4]

Correct. first takes the initial element, 1, and the rest pattern collects everything that remains afterward into a new array, 2, 3 and 4.

B. [1,2,3,4]

Assumes the rest pattern collects the entire original array alongside the named binding. The rest pattern only collects whatever elements are left over after the earlier named bindings have taken their share.

C. [3,4]

Drops one extra element from the front, as if two positions were named instead of one. Only first is named here, so rest correctly starts collecting from the second element onward.

D. [1]

Swaps the roles, as if the rest pattern captured only what first took. first binds to a single value taken from the front, while rest always collects everything that remains after it.

A rest element in a destructuring pattern must come last, and it gathers every remaining value that was not already claimed by an earlier named binding into a genuine new array. This mirrors rest parameters in function definitions, which collect any extra arguments beyond the named ones.

11. Correct answer, C

A. ReferenceError

Assumes id is never declared because the pattern only introduces bindings for data and user.

B. `{'id':7}`

Assumes the innermost pattern binds the whole user object rather than just its id property.

C. (correct) 7

Correct. Nested destructuring walks down through data and user and binds id to the number stored three levels deep.

D. undefined

Assumes destructuring through several nested levels in one pattern is not valid and silently produces undefined.

Object destructuring can nest arbitrarily deep, each colon steps into a property of the previous level. Only the innermost names actually become variables, the intermediate names like data and user are just path segments.

12. Correct answer, A

A. (correct) a

Correct. The nested array pattern takes the first element of list, which is the string a.

B. `["a","b"]`

Assumes list, once renamed inside the pattern, ends up holding the whole array instead of just its first element.

C. undefined

Assumes mixing object and array destructuring in a single pattern is not supported.

D. b

Assumes an array pattern nested inside an object pattern takes the last element rather than the first.

Object and array destructuring patterns can be nested inside each other freely. An array pattern used as the value of an object property deconstructs the array found at that property using ordinary positional rules, starting from index 0.

13. Correct answer, C

A. undefined

Assumes the comparison silently fails to produce a boolean when neither side is a named variable.

B. `TypeError`

Assumes comparing two freshly created object literals directly is not valid syntax.

C. (correct) false

Correct. Strict equality on two objects compares references, and these are two distinct object literals, so the comparison is false.

D. true

Assumes two object literals with identical contents are considered equal by strict equality.

Strict equality between two objects never inspects their properties, it only checks whether both sides refer to the exact same object in memory. Two separate object literals, even with identical shape and values, are always two different objects.

14. Correct answer, C

A. start sync-end timeout microtask

Assumes a zero delay timer fires before the microtask queue drains. Every microtask always runs before the next macrotask, no matter how short the delay.

B. start timeout microtask sync-end

Assumes the timer callback preempts the rest of the current script. The synchronous code always finishes before any timer or microtask gets a turn.

C. (correct) start sync-end microtask timeout

Correct. The synchronous code runs to completion first, then the whole microtask queue drains, and only then does the first macrotask fire.

D. start microtask sync-end timeout

Assumes an already resolved promise runs its callback immediately in place. A then callback is always deferred to the microtask queue, even on a settled promise.

JavaScript finishes the current synchronous run to completion, then empties the microtask queue completely, then runs exactly one macrotask before repeating the cycle. Promise callbacks and async continuations are microtasks, while setTimeout callbacks are macrotasks, so a promise always wins the race against a timer of the same delay.

15. Correct answer, A

A. (correct) 2

Correct. An async function used as a then handler still returns a promise, which the chain automatically adopts before continuing.

B. 1

Assumes the handler returns its input unchanged because the addition happens after an await inside it. The handler still computes and returns the incremented value once its internal await resumes.

C. [object Promise]

Assumes returning from an async then handler leaves the chain holding an unresolved promise instead of a plain number. Then always waits for a promise returned by a handler to settle before passing its value onward.

D. undefined

Assumes using an async function as a then handler prevents the chain from ever continuing at all. The chain continues normally, it simply waits one extra microtask tick for the async handler to finish.

A then handler can be an async function without anything special happening beyond the fact that it, like every async function, returns a promise. The chain treats that returned promise exactly as it would any other returned promise, waiting for it to settle before handing its value to the next handler.

16. Correct answer, B

A. caught:undefined

Assumes the error thrown by the await loses its message once it reaches the catch block. The catch block receives the original Error object with its message intact.

B. (correct) caught:mid-fail

Correct. Awaiting a Promise.all that rejects throws inside the try block, and the catch receives that exact rejection.

C. never

Assumes awaiting a Promise.all that contains a rejection still resolves normally, letting execution continue past it. A rejection anywhere in the array makes the whole Promise.all reject, which await then turns into a thrown exception.

D. caught:1,mid-fail,3

Assumes the catch handler somehow receives every value from the array combined into one string. Promise.all rejects with the single reason from the promise that actually rejected, not with the other resolved values.

A rejected Promise.all behaves like any other rejected promise when awaited, it throws at the await expression with the same reason the combined promise rejected with. A surrounding try catch handles that throw exactly as it would a rejection from a single promise, and the other successfully resolved entries are simply not part of the rejection reason.

17. Correct answer, C

A. 7

Confuses the shift operator with addition, adding the shift amount to the original number.

B. 10

Assumes each left shift multiplies by 2 regardless of the shift count, applying the doubling only once instead of the two times requested.

C. (correct) 20

Correct. Shifting left by 2 multiplies the value by four, since each position shifted doubles it.

D. 25

Assumes each left shift multiplies by 5, confusing the shift count with the value being shifted.

Shifting an integer left by n bit positions is equivalent to multiplying it by two to the power of n . Shifting 5 left by 2 multiplies it by four, giving 20.

18. Correct answer, A

A. (correct) o

Correct. The `at` method supports negative indices, and `-1` refers to the last character of the string.

B. H

Assumes a negative index to `at` counts from the start of the string instead of from the end.

C. undefined

Assumes a negative index is out of range and produces undefined, but `at` is specifically designed to support negative indices counting from the end.

D. olleH

Confuses `at` with reversing the whole string instead of indexing a single character from the end.

The `at` method was added to let negative indices count backward from the end of a string or array, unlike bracket indexing, which only accepts non negative positions. Index `-1` with `at` always refers to the very last character.

19. Correct answer, B

A. TimeoutError

Confuses the return value with the constructor name of the error object passed in.

B. (correct) slow

Correct. `instanceof` checks the exact prototype chain of the passed object, and only the `TimeoutError` branch matches it.

C. unknown

Assumes `instanceof` only matches the base `Error` class and not user defined subclasses, but a class extending `Error` still satisfies `instanceof` for itself.

D. missing

Assumes both custom classes are indistinguishable subclasses of `Error` and the first branch always wins, but `instanceof` checks the specific prototype chain of the object being tested.

`instanceof` walks the prototype chain of the object and checks each link against the given constructor. A `TimeoutError` instance is only an instance of `TimeoutError` and `Error`, never of an unrelated sibling class.

20. Correct answer, A

A. (correct) 50

Correct. The caught value is the plain number 42, and adding 8 to it gives an ordinary numeric result.

B. 42

Assumes the addition never runs because the caught value is not an Error instance, but any thrown value reaches the catch block and can be used normally.

C. NaN

Assumes a thrown non Error value cannot be combined with a number and coerces into something unusable, but a thrown number behaves exactly like any other number.

D. undefined

Assumes the caught value is undefined because the thrown target was not an Error object, then concatenates it with the second operand as a string.

Throwing a primitive value like a number delivers that exact value to catch. Arithmetic on it proceeds normally, there is no requirement that a thrown value be an Error for it to be usable afterward.

That was 20 of the 600 JavaScript questions.

The full bank is one PDF laid out exactly like this sample, all 600 questions first and the complete answer key after, with a reason for each of the four options. Every snippet in it was run the same way before its answer was written.

Get it for 9 euros at <https://ko-fi.com/s/e564f7a936>, or JavaScript, Python and SQL in one PDF for 19 euros at <https://ko-fi.com/s/fa83c4ed59>.

The same 20 questions can be played in a browser at <https://thibaudlepan77-svg.github.io/interview-questions-verified/>. If an answer looks wrong to you, open an issue on the repository linked from that page with the question number and what your own run printed.